Version 1.2

January 2026

DATA MINING

MODULE 3 - DATA PREPROCESSING

Authored by:

Yufis Azhar, Ir., S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS **2**
INTRODUCTION **3**
PRACTICAL OBJECTIVES 3
PREPARATION 3
KEYWORDS 3
1. Introduction to Data Preprocessing **4**
1.1 Basic Concepts: Why Data Preprocessing Matters 4
1.2 Characteristics of Real-World Data 4
1.3 Setup: Loading Datasets and Library Introduction 4
2. Data Cleaning **5**
2.1 Handling Missing Values 5
2.2 Handling Duplicates 7
2.3 Outlier Detection 7
3. Encoding Categorical Data **8**
3.1 Types of Categorical Data 8
3.2 Label Encoding 8
3.3 One-Hot Encoding 9
4. Normalization & Feature Scaling **10**
4.1 The Importance of Scaling 10
4.2 Min-Max Normalization 10
4.3 Standardization (Z-Score Scaling) 11
4.4 When to use which? 12
CODELAB (20%) **13**
LAB ASSIGNMENT (80%) **15**
GRADING CRITERIA & DETAILS **17**

INTRODUCTION

PRACTICAL OBJECTIVES

1. Load and inspect a dataset using Python
2. Identify missing values in the dataset
3. Apply appropriate techniques to handle missing values
4. Perform data normalization using Min-Max normalization and Z-score standardization
5. Compare the dataset before and after preprocessing

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Hardware:
   - Laptop/Computer
3. Software (Choose one):
   - Visual Studio Code, Google Collaboratory, PyCharm, Jupyter Notebook
4. Tutorial
   - Using Google Colab: https://youtu.be/tpDjhSzJor4?si=bFjSP_mf3oB5x520
   - Using Jupyter in Visual Studio Code: https://youtu.be/HOg13z1wI1A?si=RhEY2QY5bir-dzEH
   - Using Jupyter Notebook: https://youtu.be/HQaCF2F_MRo?si=k6ubuGuCg-VS0tQe
   - Install Anaconda: https://youtu.be/XyudQ3B3OWI?si=7qugN_vTxAM30qDt
5. Source Code (Google Colaboratory):
   - *https://colab.research.google.com/drive/1vOqzH3bASzBpnD9miBUJkyrcMCzivKpC#scrollTo=LGFeu9S4slaO*
6. Access Dataset:
   - Titanic Dataset
   - Pima Indian Dataset
   - Horse Colic Dataset
   - Credit Card Approval Dataset

7. Strong motivation and commitment to learning 🤗

KEYWORDS

Python Programming, Data Preprocessing, Data Cleaning, Encoding Data, Normalization

1. Introduction to Data Preprocessing

1.1 Basic Concepts: Why Data Preprocessing Matters

The performance of a Machine Learning model depends heavily on the quality of the data used to train it. In data science, there is a famous principle called "Garbage In, Garbage Out" (GIGO). This means that if you feed an algorithm poor-quality, noisy, or "dirty" data, the resulting model will produce unreliable or "garbage" predictions, no matter how advanced the algorithm is.

Data Preprocessing is the process of transforming raw data into a clean and organized format that is suitable for machine learning models. It ensures that the model can learn patterns effectively without being misled by errors or inconsistencies.

1.2 Characteristics of Real-World Data

In a classroom or competition, data is often "clean." However, in the real world, data is rarely ready for immediate use. You will often encounter:

1. **Incomplete Data**: Missing values for certain attributes (e.g., a customer's age or phone number is missing).
2. **Noisy/Dirty Data**: Containing errors, outliers, or values that don't make sense (e.g., a "Weight" of -50 kg).
3. **Inconsistent Data**: Differences in naming conventions or formats (e.g., "U.S.A" in one row and "United States" in another, or different date formats).

1.3 Setup: Loading Datasets and Library Introduction

Google Colab is an excellent environment for preprocessing because it comes with pre-installed data science libraries. The two most essential libraries are:

1. **NumPy**: Used for high-performance scientific computing and array manipulation.
2. **Pandas**: The primary tool for data manipulation and analysis (working with DataFrames).

Technical Implementation in Colab First, we must import the libraries:

Example:

```
import pandas as pd
import numpy as np

print("Pandas version:", pd.__version__)
print("NumPy version:", np.__version__)
```

Output:

```
•••  Pandas version: 2.2.2
     NumPy version: 2.0.2
```

In Google Colab, you can load data from various sources. The most common way is using a URL or uploading a local CSV file. We will use the [Titanic Dataset](#) (If you want to access via Drive, click [here](#)) for this practice.

Example:

```
# Loading a CSV dataset from a URL
url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

# Display the first 5 rows of the data
df.head()
```

Output:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

To understand how "dirty" our data is, we use the following commands:

```
# Check the structure, data types, and missing values
df.info()

# Summary statistics for numerical columns
df.describe()
```

2. Data Cleaning

2.1 Handling Missing Values

After inspecting the data in Chapter 1, you likely noticed that some columns have fewer entries than others. This indicates Missing Values. We have three main strategies to handle them:

1. **Detection**: Identifying which columns contain null values.

2. **Deletion**: Removing rows or columns with missing data (used if the missingness is high).
3. **Imputation**: Filling in the gaps with statistical values like Mean, Median, or Mode.

Example:

```
# 1. Detect missing values
print("Missing values per column:")
print(df.isnull().sum())

# 2. Imputation: Fill 'Age' with Median
# We use median because age often has outliers
df['Age'] = df['Age'].fillna(df['Age'].median())

# 3. Deletion: Drop 'Cabin' column
# We drop it because more than 70% of the data is missing
df.drop(columns=['Cabin'], inplace=True)

# 4. Fill 'Embarked' with Mode (the most frequent value)
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])

print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

Output:

```
Missing values per column:
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

```
Missing values after cleaning:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

2.2 Handling Duplicates

Duplicate data can bias your model by giving too much weight to specific observations. It is essential to identify and remove them to ensure data integrity.

Example:

```
# Check for duplicate rows
duplicate_count = df.duplicated().sum()
print(f"Number of duplicate rows: {duplicate_count}")

# Remove duplicates (if any)
df.drop_duplicates(inplace=True)
```

Output:

```
Number of duplicate rows: 0
```

2.3 Outlier Detection

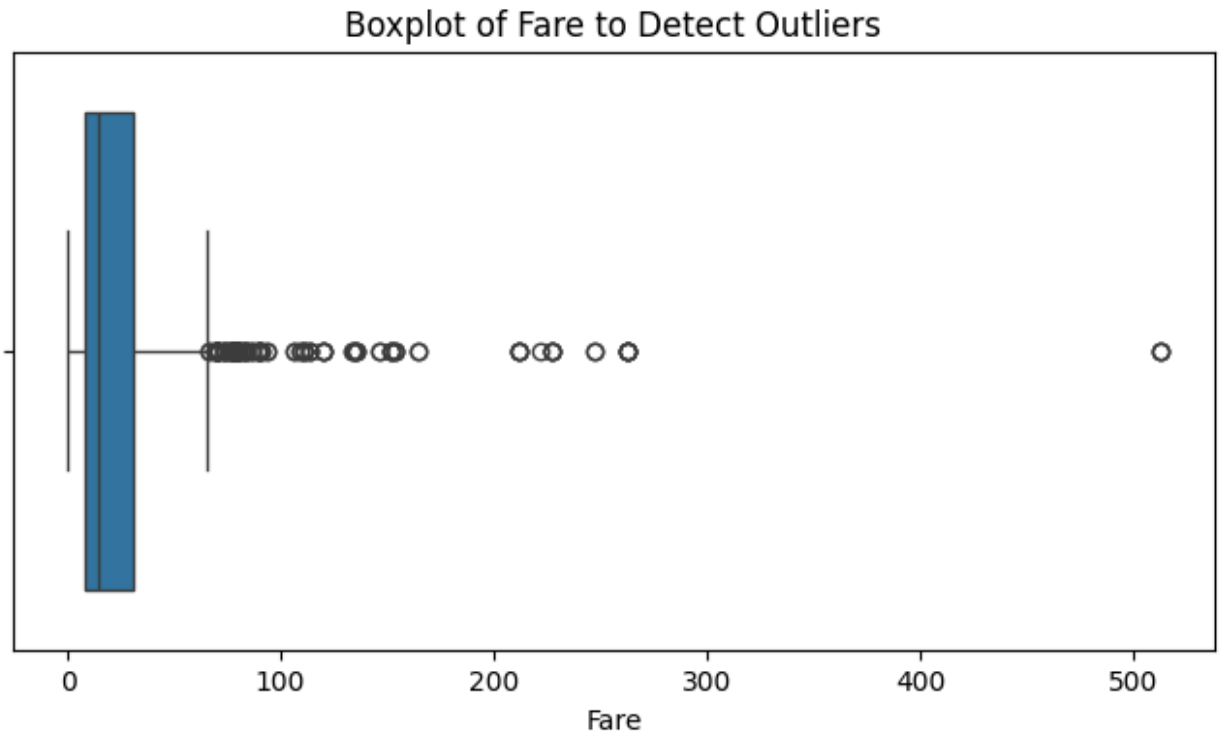
Outliers are data points that differ significantly from other observations. They can be caused by measurement errors or genuine extreme variations. In Python, we often use the Interquartile Range (IQR) method or visual tools like Boxplots.

Example:

```
import seaborn as sns
import matplotlib.pyplot as plt

# Visualizing outliers in the 'Fare' column
plt.figure(figsize=(8,4))
sns.boxplot(x=df['Fare'])
plt.title('Boxplot of Fare to Detect Outliers')
plt.show()

# Statistical Detection using IQR
Q1 = df['Fare'].quantile(0.25)
Q3 = df['Fare'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
```

```
outliers = df[(df['Fare'] < lower_bound) | (df['Fare'] >
upper_bound)]
print(f"Number of outliers detected in 'Fare': {len(outliers)}")
```

Output:




```
Number of outliers detected in 'Fare': 116
```

3. Encoding Categorical Data

3.1 Types of Categorical Data

Before encoding, we must distinguish between two types of categorical data:

1. **Nominal Data**: Categories with no inherent order (e.g., Sex: Male, Female; Embarked: S, C, Q).
2. **Ordinal Data**: Categories with a logical rank or order (e.g., Education: High School < Bachelor < Master; Satisfaction: Low < Medium < High).

3.2 Label Encoding

Label Encoding assigns a unique integer to each category (e.g., 0, 1, 2). This is best suited for Ordinal Data because it preserves the order. If used on nominal data, the model might mistakenly think that a higher number is "better" or "larger."

Example:

```
from sklearn.preprocessing import LabelEncoder

# Initialize the encoder
le = LabelEncoder()

# Example: Encoding 'Sex' (though it is nominal, often binary
labels 0/1 are used)
df['Sex_Encoded'] = le.fit_transform(df['Sex'])

# Display mapping
print("Classes defined by LabelEncoder:", le.classes_)
df[['Sex', 'Sex_Encoded']].head()
```

Output:

```
Classes defined by LabelEncoder: ['female' 'male']
```

	Sex	Sex_Encoded
0	male	1
1	female	0
2	female	0
3	female	0
4	male	1

3.3 One-Hot Encoding

For Nominal Data, we use One-Hot Encoding. This technique creates new binary columns (0 or 1) for each category. This prevents the model from assuming a mathematical hierarchy between categories like "City" or "Color."

Example:

In Pandas, we can easily do this using the `get_dummies()` function.

```
# One-Hot Encoding for 'Embarked' column
# drop_first=True is used to avoid the "Dummy Variable Trap"
(multicollinearity)
df_encoded = pd.get_dummies(df, columns=['Embarked'],
drop_first=True)
```

```
# Display the new columns
print("New columns after One-Hot Encoding:")
print(df_encoded.columns[-3:])  # Display the last 3 columns created
df_encoded.head()
```

Output:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Sex_Encoded	Embarked_Q	Embarked_S
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	1	False	True
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	0	False	False
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	0	False	True
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	0	False	True
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	1	False	True

4. Normalization & Feature Scaling

4.1 The Importance of Scaling

Imagine you are comparing two passengers:

1. Passenger A: Age 20, Fare 10.
2. Passenger B: Age 50, Fare 500.

If we don't scale, the algorithm will think the difference in Fare is much more important than the difference in Age simply because the numbers are larger. Scaling brings all features into a similar range so the model treats them fairly.

4.2 Min-Max Normalization

Normalization scales the data into a fixed range, usually 0 to 1. This is very useful when you know the distribution of your data does not follow a Gaussian (Normal) distribution.

Formula:

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Example:

```
from sklearn.preprocessing import MinMaxScaler

# Initialize the scaler
scaler = MinMaxScaler()

# We scale 'Age' and 'Fare'
df_encoded[['Age',                    'Fare']]                    =
scaler.fit_transform(df_encoded[['Age', 'Fare']])

print("Data after Min-Max Normalization (Range 0-1):")
df_encoded[['Age', 'Fare']].describe()
```

Output:

```
Data after Min-Max Normalization (Range 0-1):
```

	Age	Fare
count	891.000000	891.000000
mean	0.363679	0.062858
std	0.163605	0.096995
min	0.000000	0.000000
25%	0.271174	0.015440
50%	0.346569	0.028213
75%	0.434531	0.060508
max	1.000000	1.000000

4.3 Standardization (Z-Score Scaling)

Standardization transforms data so that it has a mean of 0 and a standard deviation of 1. This is the preferred method for many algorithms (like Linear Regression or Logistic Regression) that assume the data follows a Normal/Gaussian distribution.

Formula:

$$Z = \frac{X - \mu}{\sigma}$$

Example:

```
from sklearn.preprocessing import StandardScaler

# Initialize the scaler
std_scaler = StandardScaler()

# Applying standardization
df_std = df_encoded.copy()
df_std[['Age', 'Fare']] = std_scaler.fit_transform(df_std[['Age',
'Fare']])

print("Data after Standardization (Mean ~0, Std ~1):")
print(f"Mean Age: {df_std['Age'].mean():.2f}")
print(f"Std Age: {df_std['Age'].std():.2f}")
```

Output:

```
Data after Standardization (Mean ~0, Std ~1):
Mean Age: 0.00
Std Age: 1.00
```

4.4 When to use which?

Normalization: Use when your data has a bounded range (like image pixels 0-255) or when you use algorithms like Neural Networks or KNN. Standardization: Use when your data follows a Bell Curve (Normal distribution) or when you want to handle outliers more gracefully than Min-Max scaling.

CODELAB (20%)

Dataset (Choose One):

- Via URL: Pima Indians Diabetes
- Via Google Drive: Pima Indian Diabetes (Drive)

Focus: Handling Hidden Nulls, Outliers, and Scaling

Task 1: Data Acquisition & Inspection

Load the dataset and assign the appropriate column names. Display the first 10 rows to understand the initial structure.

- Dataset URL: Pima Indians Diabetes
- Columns: `'Pregnancies'`, `'Glucose'`, `'BloodPressure'`, `'SkinThickness'`, `'Insulin'`, `'BMI'`, `'DiabetesPedigree'`, `'Age'`, `'Outcome'`

Task 2: Identifying Hidden Missing Values

In this dataset, a value of 0 in certain columns is medically impossible and represents a Missing Value.

- Convert all 0 values to NaN only for these columns: `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, and `BMI`.
- Count and display the total number of missing values per column.

Task 3: Handling Missing Values (Imputation)

Now that you have identified the NaN values, you must fill them.

- Perform Imputation using the Median value for all columns containing NaN.
- Verify that there are no more missing values left in the dataframe.

Task 4: Outlier Detection

Extreme values in medical data (like Insulin levels) can skew model results.

- Use a Boxplot to visualize the distribution of the Insulin column.
- Briefly identify if there are many data points beyond the upper whisker.

Task 5: Feature Scaling (Normalization)

The features have very different ranges (e.g., Age vs Insulin). We need to normalize them.

- Apply Min-Max Normalization to the entire dataset.
- Ensure every value in the dataframe is now between 0 and 1.

Final Submission Check

Run the code below to ensure your data is ready for Machine Learning. All types should be numeric, and all values should be between 0 and 1.

```
# FINAL CHECK
print("--- Data Info ---")
df.info()

print("\n--- Data Descriptive Statistics ---")
display(df.describe().loc[['min', 'max']])

print("\n--- Final Data Preview ---")
df.head()
```

**This is mandatory and must not be replaced under any circumstances.*

LAB ASSIGNMENT (80%)

Dataset A (For ODD Student ID/NIM: 1, 3, 5, 7, 9)

- **Topic**: Horse Colic Medical Data
- **Dataset (Choose One)**:
  - Via URL: Horse Colic Medical Data
  - Via Google Drive: Horse Colic Medical Data (Drive)
- **Notes**: This dataset uses ? to represent missing values. You must handle this during loading. We will focus on the first 8 columns for this exercise.
- **Column Names**: `'surgery'`, `'age'`, `'id'`, `'temp'`, `'pulse'`, `'respiratory'`, `'extremity_temp'`, `'peripheral_pulse'`

Dataset B (For EVEN Student ID/NIM: 0, 2, 4, 6, 8)

- **Topic**: Credit Card Approval
- **Dataset (Choose One)**:
  - Via URL: Credit Card Approval
  - Via Google Drive: Credit Card Approval (Drive)
- **Notes**: This dataset also uses ? for missing values and contains a mix of continuous and categorical attributes.
- **Column Names**: `'A1'`, `'A2'`, `'A3'`, `'A4'`, `'A5'`, `'A6'`, `'A7'`, `'A8'`, `'A9'`, `'A10'`, `'A11'`, `'A12'`, `'A13'`, `'A14'`, `'A15'`, `'A16'`

Instructions!

Task 1: Data Acquisition & Initial Cleaning

Load your respective dataset. Ensure that the symbol "?" is correctly recognized as a Missing Value (NaN) during or after loading. Display the first 10 rows and the data types of each column.

Task 2: Advanced Imputation

1. Identify which columns are numerical and which are categorical.
2. For Numerical columns: Fill missing values with the Median.
3. For Categorical columns: Fill missing values with the Mode (most frequent value).

Task 3: Outlier Handling

1. Choose one continuous numerical column from your dataset (e.g., pulse for Dataset A or A2 for Dataset B).
2. Visualize the distribution using a Boxplot.
3. Identify if outliers exist.

Task 4: Categorical Encoding

Machine learning models require numerical input.

1. Pick one categorical column.
2. Apply Label Encoding if the data is ordinal, or One-Hot Encoding if it is nominal.

Task 5: Normalization

Apply Min-Max Normalization to all numerical columns so that the values are constrained between 0 and 1.

GRADING CRITERIA & DETAILS

Activity & Details	Weight
Codelab	**20%**
Task 1: Data Acquisition & Inspection	4%
Task 2: Identifying Hidden Missing Values	4%
Task 3: Handling Missing Values (Imputation)	4%
Task 4: Outlier Detection	4%
Task 5: Feature Scaling (Normalization)	4%
Lab Assignment	**80%**
Task 1: Data Acquisition & Initial Cleaning	6%
Understanding of Task 1	5%
Task 2: Advanced Imputation	6%
Understanding of Task 2	5%
Task 3: Outlier Handling	6%
Understanding of Task 3	10%
Task 4: Categorical Encoding	6%
Understanding of Task 4	10%
Task 5: Normalization	6%
Understanding of Task 5	20%
Total	**100%**